Práctica 3: CWS1

Objetivos

- Separar el código en dos capas: interfaz de usuario y lógica de negocio (que en esta práctica incluirá lógica de negocio y acceso a datos).

Material necesario

En el Campus Virtual de la asignatura puede encontrar:

- Un fichero comprimido cws0.zip. Contiene un proyecto Maven con módulos **cws0**, **cws_util**., **cws_contracts_payrolls_acceptance_tests**. El proyecto cws_util tiene clases de utilidad, que pueden usarse libremente pero no debe modificarse.
- Carpeta con base de datos hsqldb **cws_full_database**.
- Módulo **jdbc_cws_service_interfaces** con el código de las interfaces de servicio de uso obligatorio a medida que se necesiten. Incluyen los Data Transfer Objects (DTOs) que se utilizan como medio para pasar información entre distintas capas.

La versión de Java utilizada será la instalada en los laboratorios de prácticas.

Vamos a trabajar con la base de datos en modo fichero, por lo que no es necesario iniciarla como un servicio. Esto implica que, al cerrar la aplicación, los cambios realizados en la base de datos no se conservarán. Si desea modificar este comportamiento, basta con actualizar el parámetro URL en el fichero *configuration.properties*.

IMPORTANTE: a no ser que se indique explícitamente, está prohibido renombrar clases, paquetes, métodos, etc. Esto aplica tanto a esta sesión como a sesiones futuras relacionadas con el CarWorkShop. Tampoco está permitido mover clases a un paquete diferente del declarado en su sentencia "package" o modificar la jerarquía de clases implícita en las diferentes sentencias import del código proporcionado.

Cree un espacio de trabajo e importe en él el proyecto Maven. A partir de este código inicial debe comenzar a implementar las prácticas. Nos centraremos en el caso de uso de gestión de mecánicos, cuyo actor es el manager (gerente).
Después, ejecute la aplicación de gestión de mecánicos, a partir de la clase MainMenu en el paquete manager.

Ejercicio 1: Aplicación del patrón layer

Se separa el código relativo a la interacción con el usuario del resto del código, considerando inicialmente el caso de uso de gestión de mecánicos. Crear una estructura de paquetes para contener el código de la aplicación, separando en capas según el patrón **layer**; se añaden paquetes en función de las capas en que vamos a dividir de la aplicación.
El paquete **uo.ri.cws.application** contendrá ahora dos subpaquetes:

- **ui**. Se mantiene el paquete ui que contendrá clases que implementen la interacción con el usuario.
- **service**. Contendrá clases que implementen tanto la lógica de negocio como el acceso a la base de datos.

1.1 Organización de la capa service

La capa **service** se organiza, **por entidades del modelo del dominio para incrementar la cohesión**.

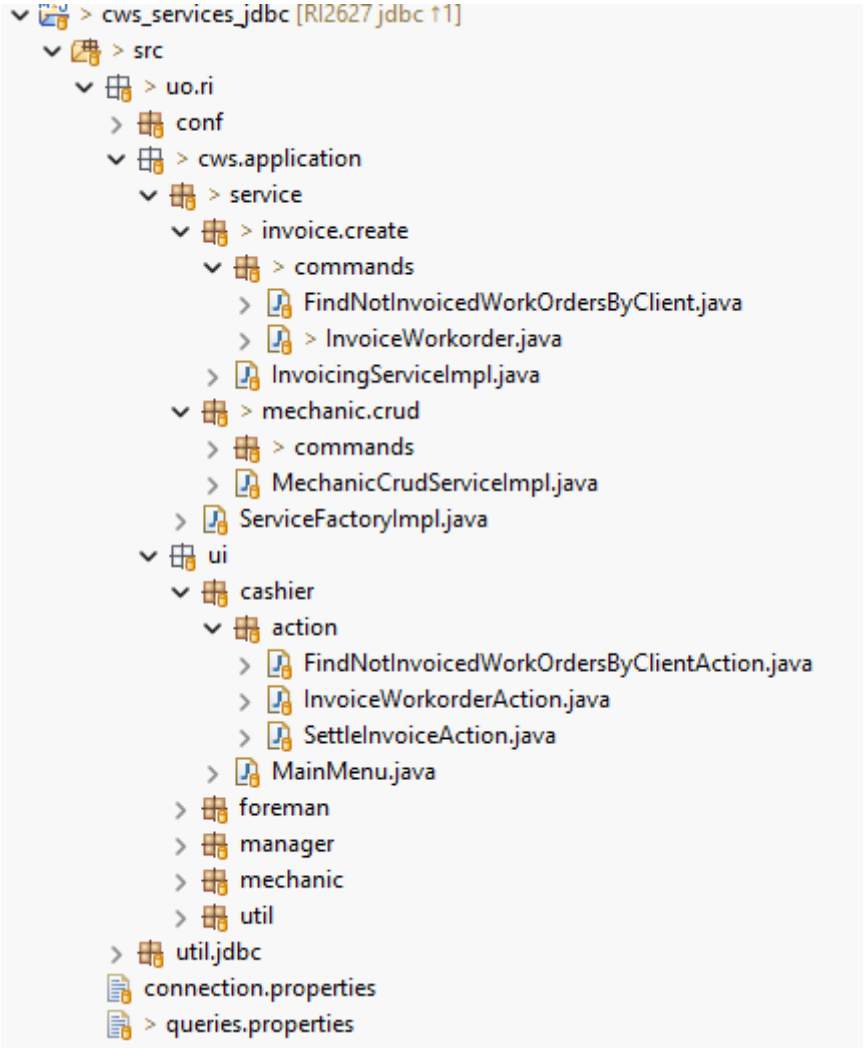
- Se creará un paquete para cada entidad del dominio (mechanic e invoice inicialmente). Cada paquete se ocupará del caso de uso de gestión de la entidad correspondiente.
- Cada uno de estos paquetes incluirá uno o varios paquetes en los que se implementarán las clases que se encargarán de llevar a cabo las operaciones del caso de uso.

  Norma de estilo: Para aquellos casos de uso que sólo tengan operaciones de creación, actualización, recuperación o borrado, como gestión de mecánicos, se añadirá un subpaquete que se llamará **crud**.

- El paquete **crud** contendrá la implementación de la interfaz de servicio.
  - La interfaz está en el proyecto **cws_service_interfaces** junto con los objetos de transferencia de datos (*DTO*, *Data Transfer Objects*), utilizados para encapsular los parámetros de entrada y salida que se intercambian entre este servicio y sus clientes.
- Dentro de este paquete, se creará un paquete **commands** que contendrá las clases que realizan las acciones de negocio. Estas clases se ajustarán, en general, a los siguientes requisitos:
  - Se llamarán igual que las clases del paquete ui, eliminando el sufijo Action.
  - Tendrán un constructor **sin parámetros o con un único parámetro**:
    - Para **añadir** o **actualizar**, un DTO con los datos.
    - Para **eliminar**, un String con el identificador del objeto.
    - Dependiendo del criterio de recuperación, podrá o no tener argumentos.

    IMPORTANTE: El constructor también debe hacer la comprobación de parámetros y lanzar IllegalArgumentException en caso necesario. Si no se indica lo contrario, parámetros vacíos, con contenido en blanco o null son considerados inválidos. Existe una clase en el proyecto cws_util para realizar estas validaciones (assertion.ArgumentChecks.java).
  - Un único método público ***execute***, sin argumentos, que **implementará las reglas de negocio,** y cuyo valor de retorno será:
    - Si se añadió: Un DTO completo, que incluya el identificador del objeto añadido.
    - Si se borró o actualizó: Void.
    - Si se leyeron (búsqueda): un DTO o una colección de DTOs, según la naturaleza del método.

Al finalizar esta práctica, la organización de paquetes será la siguiente:



1.2 Implementación de la capa service

- **Extraer de las clases ui.manager.action.XXX** todo aquel código que no se refiera a interacción con el usuario o pequeñas comprobaciones de validez de datos (longitud del campo nif, estructura del campo nif o email, etc) y mover ese código relativo a lógica de negocio o acceso a datos a las **nuevas clases del paquete service.mechanic.crud.commands**.
- Las clases de la **interfaz de usuario invocarán** ahora a las clases necesarias en **la capa de negocio** pasando como parámetro los datos que necesiten (en forma de dto, String).
- Además de implementar la lógica de negocio necesaria para llevar a cabo una acción, el método **execute()** de las nuevas clases también deberían realizar validación de reglas de negocio. *Por el momento*, se ignorarán posibles violaciones de las reglas de negocio, como añadir un mecánico repetido o borrar uno que no exista. Existe una clase uo.ri.util.assertion.BusinessChecks que realizan comprobaciones y lanzan la excepción BusinessException (paquete uo.ri.util.exception). **Recuerda implementar estas comprobaciones al final de la sesión.**

1.3 Implementación de la capa ui

- Para imprimir los mecánicos (u otras entidades del modelo de dominio), se debe usar la clase **Printer** proporcionada en el paquete ui.util.

1.4 Prueba de funcionamiento

Arranque la base de datos y realice las siguientes **pruebas**

a. Listar los mecánicos.
b. Listar un mecánico.
c. Crear un nuevo mecánico
d. Modificar el mecánico anterior
e. Borrar el mecánico anterior
f. Intentar modificar un mecánico cuyo id no existe
g. Intentar borrar un mecánico que no existe

2 Ejercicio 2: Aplicación del patrón fachada para desacoplar el código de las capas ui y service

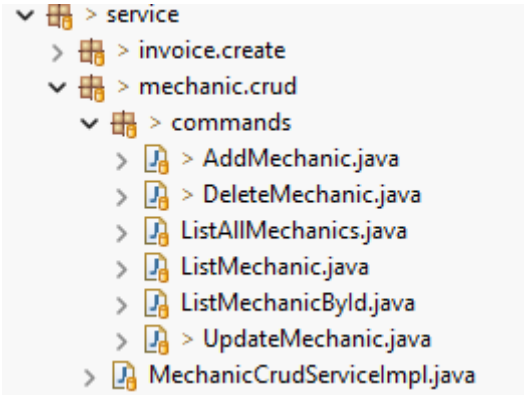
Para **eliminar dependencias** entre la capa de interfaz de usuario y la de negocio, se aplica **inicialmente** el patrón **Fachada**, dotando a cada subsistema (gestión de mecánicos u otro) de una **interfaz** (**fachada**) que actúa como **punto único de entrada**. Los clientes (**capa ui u otros clientes**) invocan métodos de la fachada; no acceden directamente a los objetos del subsistema (disminuye el acoplamiento). Tenga en cuenta estas indicaciones para introducir fachadas en su proyecto:

- Las fachadas (interfaces de servicio), proporcionadas en un proyecto independiente, no se pueden modificar (salvo que se indique lo contario).
- Para el caso de uso de gestión de mecánicos, el interfaz es **MechanicCrudService**.

2.1 Organización de la capa service

- La **declaración** de las interfaces, que está completamente prohibido modificar, están en el módulo cws_service_interfaces. Es necesario **implementar** las interfaces java, en el paquete de implementación service.mechanic.crud o service.invoice.create, dependiendo del caso.
- Cada uno de sus métodos se encargará de invocar a la clase de implementación correspondiente (AddMechanic, DeleteMechanic, etc)
- **Normas de estilo:** El nombre de la clase java que implementa la fachada será siempre el mismo que el de la interfaz, añadiendo el sufijo **Impl**.

- Modifique las clases ui (***Action) para que invoquen a métodos de la fachada (uo.ri.cws.application.service.mechanic.crud.MechanicCrudServiceImpl), y no de los objetos de implementación del servicio (AddMechanic, DeleteMechanic, …).

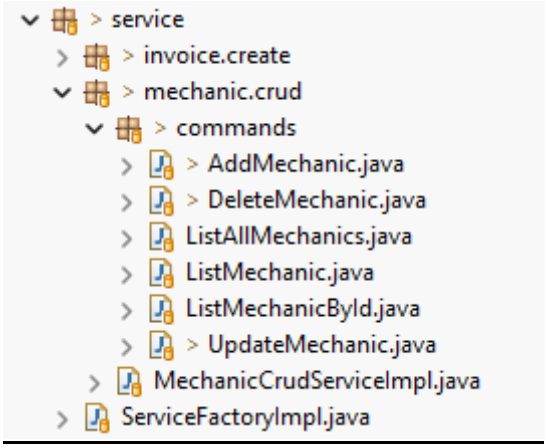


Norma de estilo: Si fuese necesario crear nuevas clases de negocio, el nombre será exactamente igual que el nombre del método de la interfaz, excepto la mayúscula inicial. Por ejemplo, un método findMechanicByAge, daría lugar a una clase FindMechanicByAge.

3 Ejercicio 3: Aplicación del patrón Simple Factory para desacoplar el código de las capas ui y service

El patrón Simple Factory (https://javajee.com/factory-design-patterns-simple-factory-factory-method-static-factory-method-and-abstract-factory) permite que se puedan obtener instancias de las fachadas sin conocer detalles cómo se crean (disminuye el acoplamiento). Una clase separada encapsula la creación de objetos.

- Se proporciona la interfaz **ServiceFactory** que ofrece métodos para obtener cada una de las fachadas anteriores (forMechanicCrudService(), forCreateInvoiceService(), etc.).
- Actuará como fábrica (factoría) de fachadas y de ella se podrán obtener **instancias** de las fachadas de los servicios sin conocer detalles de su implementación (disminuye el acoplamiento).
- Se debe implementar la interfaz mediante un objeto ServiceFactoryImpl en el paquete uo.ri.cws.service.
- Para mantener centralizadas todas las factorías (la de servicio y otras que aparecerán con posterioridad), se proporciona una clase **Factories** (otra factoría) **con objetos estáticos que hacen referencia a las diferentes factorías que puedan ir surgiendo.**
- **La interfaz de usuario utilizará la clase Factories para obtener la factoría correspondiente (service en este caso). Una vez obtenida, la factoría de servicio le proporcionará la instancia a la fachada que necesite**.
    * Modifique las clases de interfaz de usuario para eliminar las dependencias de las clases de implementación en la capa de negocio (fachadas, commands…).
    * No habrá referencias a clases (MechanicServiceImpl), sino a interfaces (MechanicService).



4 Ejercicio 4: Comprobación de las reglas de negocio.

Revise las clases en paquetes crud y compruebe que realizan la validación de las reglas de negocio como añadir un mecánico repetido o borrar uno que no exista. En ningún caso la aplicación debe lanzar excepciones al usuario.

5 Ejercicio 5: Comprobación de que la solución pasa los tests.

Ejecuta el proyecto test con tu solución. Configura el fichero pom.xml si es necesario.

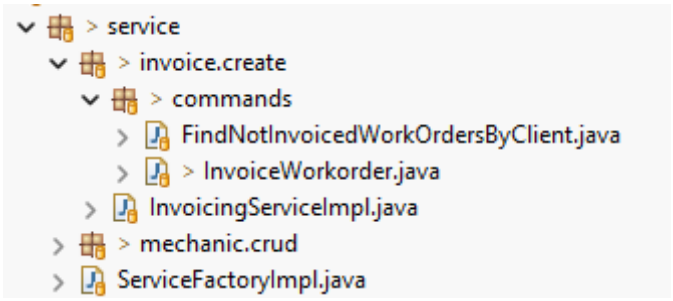
6 Ejercicio 4: Nuevos casos de uso. Facturar trabajos realizados en el taller generando la factura

A partir de la solución a los ejercicios anteriores, se debe implementar el nuevo caso de uso Facturar trabajos. Este caso de uso está ya completamente implementado en la clase InvoiceWorkorderAction en el paquete uo.ri.cws.application.ui.cashier.action.

Refactorice el código y vuelva a aplicar los patrones anteriores para eliminar de la clase InvoiceWorkorderAction todo el código que no se refiera a interfaz con el usuario, llevándolo a las clases necesarias en la capa de negocio.

Se organizará en torno a la entidad **Invoice**:

- El paquete de implementación del caso de uso será **uo.ri.cws.application.service.invoice**.
- Se provee la interfaz **InvoicingService** con el objeto de transferencia de datos **InvoiceDto**.
- Se crea un nuevo paquete **uo.ri.cws.application.service.invoice.create** para implementar el caso de uso de crear factura.
- El paquete **uo.ri.cws.application.service.invoice.create.commands** contendrá la clase CreateInvoice que implementará las acciones para facturar trabajos.



Repetir el ejercicio una vez más con el caso de uso **Buscar trabajos no facturados por cliente**. La implementación de este caso de uso se encuentra actualmente en **ui.cashier.action*.FindNotInvoicedWorkOrdersByClientAction***.